Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Dersin biçimi: öğrenci sunumları
  - Bu dersten itibaren Chris Richardson'ın "Microservices Patterns" kitabının bölümleri öğrenciler tarafından sunulur. Bu derste iki sunum yapılır: Bölüm 1 "Escaping Monolithic Hell" (Monolitik cehennemden kaçış) ve Bölüm 2 "Decomposition Strategies" (Ayrıştırma stratejileri). Hoca sunumların arasında ve sonunda özet ve düzeltme yapar.
  - İlk sunum İngilizce yapılmıştır ve ses kaydı büyük ölçüde anlaşılmaz durumdadır. Bu bölümün içeriği çoğunlukla ekrandaki slaytlardan alınmıştır.
- Bölüm 1: Monolitik mimari (ekrandan, 00:05:10)
  - Bölümün hedefleri: monolitik mimarinin (monolitik cehennem) ne olduğu, yararları ve zorlukları; mikroservis mimarisinin anlamı, yararları ve sakıncaları; mikroservis ile DevOps ilişkisi; mikroservis kalıp dili ve mimariyi tasarlamada kullanımı.
  - Monolitik mimari klasik yazılım geliştirme yoludur: bütün bileşenler tek bir kod tabanı olarak geliştirilir ve tek parça olarak dağıtılır (deploy). Sunumda tek süreçli monolit ve modüler monolit olmak üzere iki tür anılır; modüler monolitte tek uygulama süreci birden çok modülden oluşur, veritabanı ortaktır.
  - Yararları (ekrandan, 00:10:00): geliştirmesi basittir (tek uygulamaya odaklanır), uygulamada köklü değişiklik yapmak kolaydır (kodu ve veritabanı şemasını değiştir, derle, dağıt), test etmesi ve dağıtması doğrudandır, ölçeklemesi kolaydır.
  - Sorunları (ekrandan, 00:10:00): güvenilirlik (veritabanı çökerse ona bağlı bütün özellikler çalışmaz), güncelleme (tek büyük kod tabanı ve sıkı bağlılık yüzünden her değişiklikte bütün uygulama yeniden dağıtılır), teknoloji yığını (değiştirmek hem maliyet hem zaman açısından pahalıdır).
- Bölüm 1: Mikroservis mimarisi (ekrandan, 00:13:10)
  - Uygulama bir ya da daha çok kod tabanından oluşur ve bunlar bağımsız olarak dağıtılabilir. Slayttaki şekilde üç istek, her biri kendi yönlendiricisi (router), özelliği (Feature A/B/C) ve kendi veritabanı olan üç ayrı parçaya gider.
  - Facebook, Twitter gibi sosyal medya şirketlerinde ve Amazon gibi perakendecilerde kullanıldığı söylenir.
  - Yararları (ekrandan, 00:14:40): büyük ve karmaşık uygulamalarda sürekli teslim ve dağıtım; servisler küçük ve bakımı kolay; bağımsız dağıtılabilir; bağımsız ölçeklenebilir; takımlar özerk olabilir; yeni teknolojiyi denemek ve benimsemek kolay; hata yalıtımı (fault isolation) daha iyi.
  - Sakıncaları (ekrandan, 00:20:00): doğru servis kümesini bulmak zordur (daha çok servis daha çok kaynak demektir, veritabanı ve transaction yönetimi zorlaşır); dağıtık sistemler karmaşıktır (geliştirme, test ve dağıtım zorlaşır; test öncesi her bağımlı servisin doğrulanması gerekir); birden çok servise yayılan özelliklerin dağıtımı dikkatli koordinasyon ister; mikroservise ne zaman geçileceğine karar vermek zordur.
- Bölüm 1: DevOps ve mikroservis ilişkisi (ekrandan, 00:21:30)
  - DevOps, geliştiricilerle BT operasyon ekiplerinin daha yakın çalışıp daha kaliteli yazılımı daha hızlı teslim etmesini sağlayan bir yöntemdir. Geliştiricilere üretim ortamını görme imkânı verir, yazılımı küçük parçalara bölerek teslim hızını ve kaliteyi artırır.
  - Mikroservisler DevOps takımlarının bağımsız işlev parçalarını paralel geliştirmesini sağlar: yeniliği hızlandırır, takımları daha üretken yapar, hataları ve maliyeti azaltır, ürün kalitesini ve iş değerini artırır. Slayttaki sonuç: DevOps mikroservis mimarisinin

sakıncalarını azaltır.

- Bölüm 1: Mikroservis kalıp dili (ekrandan, 00:25:30)
  - Kalıp (pattern) ve kalıp dili kavramları Christopher Alexander tarafından 1977'de ortaya konmuştur.
  - Kalıp: belirli bir bağlamda ortaya çıkan bir probleme yeniden kullanılabilir çözümdür. Kalıp dili: belirli bir alandaki problemleri çözen ilişkili kalıplar topluluğudur. Mikroservis kalıp dili, geliştiricinin uygulamayı mikroservis mimarisiyle kurmasına yardım eden kalıplar topluluğudur.
  - Kalıp dilinin iki amacı: mikroservislerin uygulamaya uygun olup olmadığına karar vermeyi sağlamak ve mikroservis mimarisini başarıyla kullanmayı sağlamak.
  - Sunumda kalıplar arası ilişkiler anılır: öncül (predecessor) kalıp bu kalıba ihtiyaç doğurur, ardıl (successor) kalıp bu kalıbın getirdiği bir sorunu çözer, alternatif kalıp aynı probleme başka bir çözüm sunar.
- Bölüm 2: Yazılım mimarisi ve 4+1 görünüm modeli (ekrandan, 00:40:00 ve 00:41:40)
  - Geleneksel olarak mimarinin hedefi ölçeklenebilirlik, güvenilirlik ve güvenliktir; bugün yazılımın hızlı ve güvenli teslimini de sağlaması gerekir. Mikroservis mimarisi uygulamaya yüksek sürdürülebilirlik (maintainability), test edilebilirlik ve dağıtılabilirlik (deployability) kazandıran bir mimari stildir; uygulama bir servisler kümesidir.
  - Perry ve Wolf: yazılım mimarisi = {elemanlar, biçimler, gerekçe/kısıtlar}.
  - 4+1 görünüm modeli: mantıksal görünüm (tasarımın nesne modeli), süreç görünümü (eş zamanlılık ve senkronizasyon), fiziksel görünüm (yazılımın donanıma eşlenmesi, dağıtık yön), geliştirme görünümü (yazılımın geliştirme ortamındaki statik düzeni) ve +1 olarak kullanım durumları ya da senaryolar.
  - Gereksinimler işlevsel ve kalite gereksinimleri diye ikiye ayrılır. Örnek: kullanıcının giriş yapabilmesi işlevsel gereksinimdir; 10 saniyede giriş yapabilmesi kalite gereksinimidir.
- Bölüm 2: Mimari stiller (ekrandan, 00:46:20, 00:48:10, 00:50:00)
  - Katmanlı mimari: üç katman; sunum (kullanıcı arayüzü ya da dış API'ler), iş mantığı ve kalıcılık (veritabanıyla etkileşim). Popüler üç katmanlı mimari, katmanlı stilin mantıksal görünüme uygulanmasıdır. Sakıncası: tek sunum ve tek kalıcılık katmanı vardır, iş mantığı kalıcılık katmanına bağlıdır.
  - Altıgen (hexagonal) mimari: katmanlıya alternatiftir, iş mantığını merkeze koyar. Sunum katmanı yerine iş mantığını çağıran gelen (inbound) adaptörler, kalıcılık katmanı yerine iş mantığının çağırdığı giden (outbound) adaptörler vardır. Port, bir işlem kümesi tanımlar ve iş mantığının dışarıyla nasıl etkileştiğini belirler. Veritabanına erişim DAO (Data Access Object) adaptörüyle yapılır; böylece veritabanı değişse bile iş mantığı etkilenmez.
  - Monolitik mimari: uygulamayı tek bir çalıştırılabilir ya da WAR dosyası olarak yapılandırır. Geleneksel web uygulaması örneği: Tarayıcı - Apache - Tomcat içinde tek WAR (StoreFrontUI, Accounting, Inventory, Shipping servisleri) - MySQL. Geliştirmesi, test etmesi, dağıtması, ölçeklemesi basittir; ancak kod büyür, web kapsayıcısı aşırı yüklenir, sürekli dağıtım zorlaşır, geliştirmeyi ölçeklemek zordur ve uzun süreli bir teknoloji yığınına bağlılık gerekir.
- Bölüm 2: Servis ve gevşek bağlılık (ekrandan, 00:57:30 ve 01:00:00)
  - Servis, işe yarar bir işlev sunan, bağımsız dağıtılabilir bir yazılım bileşenidir. API'si komutlar (createOrder() gibi; işlem yapar, veriyi günceller), sorgular (findOrderById() gibi; veriyi okur) ve olaylardan (OrderCreated gibi; istemcilerin abone olduğu) oluşur. Tipik bir servis altıgen mimariye sahiptir.
  - Gevşek bağlılık: servisler arasındaki bağımlılık az olmalıdır. Yararları: daha az karşılıklı bağımlılık, daha az koordinasyon, daha az bilgi akışı, daha esnek yapı. Sürdürülebilirlik ve test edilebilirlik artar.
  - Ortak (shared) kütüphaneler kod tekrarını ve disk kullanımını azaltır, ama kütüphane değiştiğinde onu kullanan bütün servisler etkilenir.
  - Servis boyutu: "mikro" çok küçük olmak zorunda demek değildir; önemli olan işlevlerin ayrılmasıdır.
- Bölüm 2: Mikroservis mimarisini tanımlamak (ekrandan, 01:10:00, 01:12:45)
  - Mimariyi tanımlamak mekanik olarak izlenebilecek bir süreç değildir; slayttaki ifadeyle mimariyi tanımlamak bilimden çok sanattır.
  - Üç adım: (1) sistem operasyonlarını belirle, (2) servisleri belirle, (3) servis API'lerini ve iş birliklerini tanımla. Başlangıç noktası kullanıcı hikâyeleri gibi gereksinimlerdir;

bir sistem operasyonu bir dış isteği temsil eder.
  - Sistem operasyonları iki türdür: komutlar (veri oluşturan, güncelleyen, silen) ve sorgular (veri okuyan).
  - Business capability (iş yeteneği) ile ayrıştırma ve alt alan (subdomain) ile ayrıştırma (Domain-Driven Design) yaklaşımları.
- Bölüm 2: Ayrıştırma ilkeleri ve engelleri (ekrandan, 01:20:00, 01:21:15, 01:27:30)
  - Tek sorumluluk ilkesi (SRP) ve ortak kapanış ilkesi (CCP).
  - Engeller: ağ gecikmesi, senkron iletişimin erişilebilirliği düşürmesi, servisler arası veri tutarlılığı ve tanrı sınıfları (god classes).
  - Sistem operasyonlarının servislere atanması: noteUpdatedLocation() örneği (ekrandan, 01:30:00).

Hocanın Özellikle Vurguladığı Kısımlar

- Monolitin asıl sorunu tek hata noktası ve toplu dağıtımdır
  - Her şey tek yerde olduğu için hızlı geliştirilip kurulur, ama en ufak değişiklikte bütün uygulamayı yeniden dağıtmak gerekir. Tek hata noktası (single point of failure) vardır: veritabanı sunucusunun ağı kopunca ya da bir tablo bozulunca bütün uygulama çöker. Hoca, devlet dairelerinde duyulan "sistem çöktü" sorunlarının büyük kısmının bu tür eski, monolitik uygulamalardan kaynaklandığını söyler.
  - Geçen hafta görülen istemci-sunucu mimarisi de aslında monolitik bir yapı oluşturur.
- Bölmek çözümdür ama orkestrasyon problemini doğurur
  - Hoca bunu bir tabak yemeğe benzetir: tek seferde ağza atılamaz, çatal bıçakla lokmalara bölünürse rahatça yenir. Sistem işlev bazında servislere bölünür, servisler de olabildiğince bağımsız çalışıp geliştirilebilsin diye daha da bölünür.
  - Ama bölünce yeni sorular çıkar: hangi servis hangi sırayla çağrılacak, kim kimi bekleyecek, kuyruk mekanizması nasıl kurulacak, hangi veriye hangi veritabanı parçasından erişilecek, hizmet veren servisler nasıl keşfedilecek. Bunlara orkestrasyon problemleri denir; çözüm için tasarım kalıpları vardır.
- Kalıplar iyi tasarımın aracıdır
  - Hoca derste çok fazla kalıp aktarmadığını, asıl amacın tasarımın nasıl yapılacağını öğretmek olduğunu söyler. Herkes benzer alanlarda benzer problemler yaşar; biri iyi bir çözüm bulunca bu, benzer problemler için de iyi bir yol olur. Mikroservis kalıpları da bu problemleri çözmek için ortaya atılmıştır.
- Boyut değil, dağıtık monolitten kaçınmak önemlidir (01:06:12)
  - Hoca sunumu düzeltir: mikroservisin küçük olması istenir ama boyut asıl önemli şey değildir. Bir değişiklik birden çok servisin birlikte değişmesini gerektiriyorsa servis kırılımında sorun vardır; yapılan şey mikroservis değil, monolitik mimarinin dağıtık hâle getirilmesidir. Hoca, mikroservis yaptığını söyleyen pek çok kurumun aslında bunu yaptığını belirtir.
- Domain modeli önceki derslerdeki analiz modelidir
  - Yüksek seviyeli domain modeli, derste UML sınıf diyagramıyla çıkarılan analiz modelinin ta kendisidir.
- Ayrıştırma özeti (01:18:50 ve 01:31:26)
  - Önce problem anlaşılır ve modellenir (analiz modeli). Sonra işlevsel parçalara eşlenir. Bu parçaların bir kısmı daha da bölünür, bir kısmı doğrudan servis olur, birbirine çok yakın olanlar tek serviste birleşir. Amaç her servisin tekil bir sorumluluk noktası olmasıdır: alanı (domain) alt alanlara bölüp her birini belli servislere eşlemek.

Kısa Tekrar Notları

- Monolit: tek kod tabanı, tek dağıtım birimi (tek çalıştırılabilir ya da WAR). Geliştirmek, test etmek, dağıtmak, ölçeklemek basit. Sorunlar: güvenilirlik (tek hata noktası), her değişiklikte tam yeniden dağıtım, teknoloji yığınına uzun süreli bağlılık.
- Mikroservis: uygulama bağımsız dağıtılabilen servisler kümesidir. Yarar: sürekli teslim, küçük ve bakımı kolay servisler, bağımsız dağıtım ve ölçekleme, özerk takımlar, yeni teknolojiyi denemek kolay, hata yalıtımı.
- Mikroservis sakıncaları: doğru servis kümesini bulmak zor, dağıtık sistem karmaşık, çok servise yayılan özellikleri dağıtmak koordinasyon ister, ne zaman geçileceğine karar ver-

mek zor.
- DevOps geliştirme ile operasyonu yakınlaştırır; mikroservisle birlikte teslimi hızlandırır.
- Kalıp: belirli bağlamdaki probleme yeniden kullanılabilir çözüm. Kalıp dili: bir alandaki ilişkili kalıplar. Christopher Alexander, 1977.
- 4+1: mantıksal, süreç, geliştirme, fiziksel + senaryolar.
- Katmanlı: sunum, iş mantığı, kalıcılık. Altıgen: iş mantığı merkezde, inbound/outbound adaptör ve port.
- Servis API'si = komutlar + sorgular + olaylar.
- Gevşek bağlılık: az bağımlılık, az koordinasyon, az bilgi akışı, esneklik.
- Mimari tanımlamanın üç adımı: sistem operasyonları, servisler, servis API'leri ve iş birlikleri.
- Komut: oluştur, güncelle, sil. Sorgu: oku.
- SRP: "Bir sınıfın değişmek için tek bir nedeni olmalı." CCP: aynı nedenle birlikte değişen sınıflar aynı pakette olmalı; CCP dağıtık monolit anti-kalıbının panzehiridir.
- Engeller: ağ gecikmesi, senkron iletişim, veri tutarlılığı, tanrı sınıfları.
- Bir değişiklik birden çok servisi birlikte değiştirmeyi gerektiriyorsa servis kırılımı yanlıştır.

Detaylı Açıklamalar

Monolitik cehennem ve neden bölüyoruz?

Monolitik mimaride uygulamanın bütün bileşenleri tek bir kod tabanında geliştirilir ve tek parça olarak dağıtılır. Bu küçük uygulamalar için iyi bir seçimdir: tek uygulamaya odaklanıldığı için geliştirmesi basittir, kodda ve veritabanı şemasında köklü değişiklik yapıp yeniden derleyip dağıtmak kolaydır, test ve dağıtım doğrudandır, ölçeklemek için uygulamanın kopyaları çalıştırılabilir (ekrandan, 00:10:00).

Uygulama büyüdükçe sorunlar belirginleşir. Veritabanı çökerse ona bağlı bütün özellikler çalışmaz. Kod tabanı büyük ve sıkı bağlı olduğu için küçük bir değişiklik bile bütün uygulamanın yeniden dağıtılmasını gerektirir. Teknoloji yığını değiştirmek hem pahalı hem zaman alıcıdır. Hoca bunu tek hata noktası kavramıyla bağlar: veritabanı sunucusunun ağı koptuğunda ya da bir tablo bozulduğunda, sistem sorguları düzgün yapamaz ve tümüyle hata ya da tepkisizlik moduna geçer.

İlk akla gelen çözüm, tek yerde duran her şeyi bölmektir. İşlev bazında bölme, kullanıcı tepkilerini ve sistemin hizmetlerini birer servis olarak modeller. Ancak bölmenin de bir bedeli vardır: servislerin kimin kimi hangi sırayla çağıracağı, kuyruklar, veriye erişim ve servis keşfi gibi orkestrasyon problemleri. Kitabın geri kalanı bu problemleri ve önerilen çözüm kalıplarını işler.

Mikroservis mimarisinin yararları ve sakıncaları

Mikroservis mimarisinde uygulama bir ya da daha çok kod tabanından oluşur ve bunlar bağımsız dağıtılır (ekrandan, 00:13:10). Her servis küçük olduğu için bakımı ve testi kolaydır, yalnız değişen servis yeniden dağıtılır, yük gören servis ayrıca ölçeklenir, her takım kendi servisinde özerk çalışır ve bir servisteki hata bütün sistemi durdurmaz.

Bedeli dağıtık sistem karmaşıklığıdır. Doğru servis kümesini bulmak zordur; servis sayısı arttıkça kaynak tüketimi, veritabanı ve transaction yönetimi zorlaşır. Bir özelliği test etmek için bağımlı servislerin de hazır olması gerekir. Birden çok servise yayılan bir özelliği dağıtmak dikkatli koordinasyon ister (ekrandan, 00:20:00). İkinci sunumda bir örnek verilir: Java ile yazılmış her servis için ayrı bir Java sanal makinesi ayağa kalkar; servisler küçük olsa da toplam bellek tüketimi artar.

4+1 görünüm modeli ve mimari stiller

Bir yazılım mimarisini tanımlamak için birden çok görünümden oluşan bir model kullanılır (ekrandan, 00:41:40). Mantıksal görünüm nesne modelidir (sınıflar ve paketler). Süreç görünümü eş zamanlılık ve senkronizasyonu gösterir. Fiziksel görünüm yazılımın donanıma eşlenmesidir

ve sistem mühendislerinin, topolojinin alanıdır. Geliştirme görünümü yazılımın geliştirme ortamındaki statik düzenidir. Kullanım durumları ve senaryolar beşinci (+1) görünüm olabilir.

> **Ders sonrası not:** Sunumda ve slaytta +1 görünüme "animation view" denmiştir. Kruchten'in modelinde bu görünümün adı büyük olasılıkla "scenarios" (senaryolar) görünümüdür; Richardson'ın kitabında senaryoların diğer görünümleri "canlandırdığı" (animate) söylenir.

Katmanlı mimaride sunum katmanı iş mantığını, iş mantığı da kalıcılık katmanını çağırır. Tek sunum ve tek kalıcılık katmanı olması, birden çok arayüzü ve birden çok veritabanını desteklemeyi zorlaştırır. Altıgen mimari bunu çözmek için iş mantığını merkeze koyar: dışarıdan gelen istekler gelen adaptörler (ör. bir controller sınıfı, mesaj tüketicisi) üzerinden gelen portlara, iş mantığının dışarıya çağrıları giden portlar üzerinden giden adaptörlere (DAO, mesaj üreticisi) gider (ekrandan, 00:48:10). Böylece iş mantığı arayüz ve veritabanı teknolojisinden bağımsız kalır.

Servisi tanımlama: komut, sorgu ve olay

Servis API'si üç tür öğeden oluşur (ekrandan, 00:57:30). Komutlar veriyi değiştirir: restoran sisteminde sipariş oluşturmak veritabanına kayıt attığı için createOrder() bir komuttur. Sorgular yalnız okur: "siparişim ne durumda?" sorgusu findOrderById() gibi bir sorgudur. Olaylar, veri değiştiğinde yayımlanır (OrderCreated, OrderCancelled) ve istemciler bunlara abone olur.

Sistem operasyonlarını tanımlama örneği

Kitaptaki FTGO (yemek siparişi) örneğinde kullanıcı hikâyelerinden komutlar çıkarılır (ekrandan, 01:12:45): tüketici sipariş oluşturur (createOrder()), restoran siparişi kabul eder (acceptOrder(); restoranın siparişi kabul ettiğini ve belirtilen zamanda hazırlamayı üstlendiğini gösterir). createOrder() komutunun belirtimi:

```
İşlem:         createOrder(consumer id, payment method, delivery address,
               delivery time, restaurant id, order line items)
Döndürür:      orderId, ...
Ön koşullar:   - Tüketici vardır ve sipariş verebilir.
               - Sipariş kalemleri restoranın menüsündeki ürünlere karşılık gelir.
               - Teslim adresi ve zamanı restoran tarafından karşılanabilir.
Son koşullar:  - Tüketicinin kredi kartı sipariş toplamı için onaylanmıştır.
               - Sipariş PENDING_ACCEPTANCE durumunda oluşturulmuştur.
```

Bu, önceki derslerde görülen işlem belirtimlerinin (ön koşul ve son koşul) aynısıdır.

Ayrıştırma ilkeleri ve engelleri

SRP, Robert C. Martin'in "Bir sınıfın değişmek için tek bir nedeni olmalı" ilkesidir; mikroservise uygulanınca her biri tek sorumluluğu olan küçük, uyumlu servisler elde edilir, servislerin boyutu azalır ve kararlılıkları artar (ekrandan, 01:20:00). CCP'ye göre aynı tür değişikliklere karşı birlikte kapanan sınıflar aynı pakette olmalıdır; bir iş kuralı değiştiğinde ideal olarak tek bir paket, tek bir servis ve tek bir takım etkilenir (ekrandan, 01:21:15).

Ayrıştırmanın engelleri: servisler arası ağ gecikmesi (toplu (batch) API'lerle azaltılabilir), senkron iletişimin bir servisi diğerine bekletmesi (asenkron iletişim önerilir), iki servisin aynı veriyi değiştirmesinde veri tutarlılığı (birbirinin yazdığını ezme) ve tanrı sınıfları. Order sınıfı bir tanrı sınıfı örneğidir: sipariş alma, restoran, teslimat ve faturalama yönlerinin hepsinin alanlarını ve işlemlerini tek sınıfta toplar (ekrandan, 01:27:30).

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.